

1. OpenClaw Gateway User Manual

Gateway is a resident process of OpenClaw: it is responsible for maintaining connections to various message channels, hosting the control and event plane, and serving as a unified entry point for sessions, routing, and channel status. Simply put: whether you can "connect to OpenClaw", whether the console can open, whether messages can come in - in many cases, it depends on whether the Gateway is running properly.

1. OpenClaw Gateway User Manual

1. When do you need to care about the gateway
2. Fastest startup method (local)
3. What "external interfaces" does the gateway provide
4. Hot reload and "requires restart" changes
5. Authentication and remote access (most common pitfalls)
 - 5.1 Gateway authentication (token/password)
 - 5.2 Recommended way for remote access: tunnel first, then access
 - 5.3 Console won't open via plain HTTP (device identity required)
6. Multi-instance and "rescue robot" mode
7. Troubleshooting sequence, can locate most issues
 - 7.1 Check status first
 - 7.2 Gateway won't start: configuration validation failed
 - 7.3 Common phenomenon: service installed but not actually running

1. When do you need to care about the gateway

In most cases, you just need to let it run stably; you only need to specifically check/learn about the gateway in these scenarios:

- Browser console on Jetson frequently reports errors, preparing to switch to TUI or remote access
- Configuration changed but "feels like it didn't take effect", suspecting hot reload/restart wasn't triggered
- Want to expose the console to LAN or access remotely via SSH/Tailscale
- Want to do multi-instance isolation (e.g., "rescue robot"/redundancy)
- Port conflicts, process won't start, service "appears installed but not running"

2. Fastest startup method (local)

Execute on the gateway host:

```
openclaw gateway
```

Common parameters:

```
# Print more complete debug/trace information to current terminal (for
troubleshooting)
openclaw gateway --port 18789 --verbose

# When port is occupied, try to terminate the listener occupying the port and
force start
openclaw gateway --port 18789 --force
```

Port priority (from high to low): `--port` > `OPENCLAW_GATEWAY_PORT` > `gateway.port` > default `18789`.

3. What "external interfaces" does the gateway provide

The same port (default 18789) will simultaneously provide WebSocket control plane and HTTP services (control interface, hooks, A2UI, etc.), belonging to "single-port multiplexing".

Common points you'll use:

- Local console: `http://127.0.0.1:18789/`
- OpenAI Chat Completions compatible interface: `/v1/chat/completions`
- OpenResponses interface: `/v1/responses`
- Tool calling interface: `/tools/invoke`

Additionally, the gateway will also start Canvas static file service by default (default port 18793), used to provide editable HTML/A2UI resources (default served from `~/openclaw/workspace/canvas`). To disable, set `canvasHost.enabled=false` or `OPENCLAW_SKIP_CANVAS_HOST=1`.

4. Hot reload and "requires restart" changes

The gateway monitors `~/openclaw/openclaw.json` (or path specified by `OPENCLAW_CONFIG_PATH`), and configuration updates **are usually automatically applied**. Default reload mode is `gateway.reload.mode="hybrid"`: safe changes are hot-applied, critical changes trigger restart.

You can explicitly configure reload behavior:

```
{
  "gateway": {
    "reload": { "mode": "hybrid", "debounceMs": 300 }
  }
}
```

Experience recommendations:

- Changing business configurations like models, agents, routing, etc., usually doesn't require manual restart
- Changing `gateway.*` (ports, binding, authentication, TLS, HTTP, etc.) are infrastructure changes, more likely to trigger restart

5. Authentication and remote access (most common pitfalls)

5.1 Gateway authentication (token/password)

The gateway enables authentication by default: you can set `gateway.auth.token` (or `OPENCLAW_GATEWAY_TOKEN`) or `gateway.auth.password`. Clients need to carry it in `connect.params.auth.token/password` when connecting.

If you use the wizard process, it will usually generate a token by default (even if only bound to loopback).

5.2 Recommended way for remote access: tunnel first, then access

Most recommended: Tailscale/VPN. Second choice: SSH tunnel.

Example (mapping remote 18789 to local 18789):

```
ssh -N -L 18789:127.0.0.1:18789 user@host
```

Then you access locally:

- Web: `http://127.0.0.1:18789/`
- WS: `ws://127.0.0.1:18789`

Note: Even when going through a tunnel, if the gateway is configured with a token, clients still need to carry the token to connect.

5.3 Console won't open via plain HTTP (device identity required)

If you open the console with `http://<lan-ip>:18789/` on the LAN, the browser may be in a non-secure context, causing WebCrypto restrictions, making it unable to generate device identity, resulting in `device identity required` / `connect failed`. [[^gateway_troubleshooting](#)]

Priority fix routes:

- Open locally: `http://127.0.0.1:18789/`
- For remote scenarios, use Tailscale Serve to provide HTTPS
- When must use HTTP, enable `gateway.controlui.allowInsecureAuth: true` and use gateway token (token mode only, no device identity/pairing) [[^gateway_troubleshooting](#)]

6. Multi-instance and "rescue robot" mode

Only try this operation when needed

Usually one gateway per host is enough; only when you need redundancy or strong isolation (e.g., rescue robot), it's recommended to run multiple gateway instances.

The core principle of multi-instance is "full isolation + no conflicts":

- Different `gateway.port`
- Different `OPENCLAW_CONFIG_PATH`
- Different `OPENCLAW_STATE_DIR`
- Different workspaces (`agents.defaults.workspace`)

Under the dev configuration file, you can quickly start a completely isolated development instance without affecting the main environment:

```
openclaw --dev setup
openclaw --dev gateway --allow-unconfigured
openclaw --dev status
openclaw --dev health
```

7. Troubleshooting sequence, can locate most issues

7.1 Check status first

Execute in order: [^gateway_troubleshooting]

```
openclaw status
openclaw status --all
openclaw status --deep
```

Common supplementary commands:

```
openclaw gateway probe
openclaw channels status --probe
openclaw gateway status
openclaw logs --follow
openclaw doctor
```

7.2 Gateway won't start: configuration validation failed

OpenClaw configuration has strict schema validation: unknown keys, type errors, or invalid values may cause the gateway to refuse to start.

When validation fails, usually only diagnostic commands are available (e.g., `openclaw doctor / logs / health / status`). Recommend directly:

```
openclaw doctor
```

7.3 Common phenomenon: service installed but not actually running

If you install the gateway as a service using systemd/launchd/schtasks, etc., showing "loaded/installed" doesn't equal the process running. Priority check: [^gateway_troubleshooting]

```
openclaw gateway status
openclaw logs --follow
```